

TRACK 1 · AZURE DATA ENGINEERING

Azure Data Engineering ADF — No Code Pipelines

P²

ACADEMY



Pranav Kumar Reddy

Azure Data Engineer · Microsoft Certified Fabric Engineer

2

MODULES

12

SECTIONS

60+

SUBTOPICS

100+

LABS

12

PROJECTS

Zero Azure experience needed

Working professionals switching to Data Engineering

100+ hands-on labs + 12 sample projects

Job-ready from Day 1

Why Choose P² Academy?

PRACTICAL & HANDS-ON LEARNING

100% practical training on real-world ADF pipelines in live Azure environments

LATEST TOOLS & TECHNOLOGIES

Curriculum updated as per industry trends — ADF, Fabric, Databricks, Synapse

JOB-FOCUSED CURRICULUM

Build job-ready skills, resume preparation, and interview confidence

DEDICATED LAB 24/7 ACCESS

24/7 access to dedicated cloud labs for hands-on practice and experimentation

DOUBTS MENTOR FOR CLARIFICATION

Personal doubt support and mentor

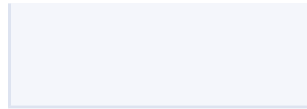
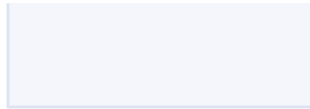
PLACEMENT ASSISTANCE

Dedicated placement support with mock interviews and referrals

LIFETIME ACCESS & COMMUNITY

Access to study materials, recordings, and exclusive student community

guidance whenever
you need it



Course Roadmap — 8 Weeks

Industry Relevant • Hands-On • Job Focused

WEEK 1

ADF Foundations

- What is ADF
- ADF Architecture
- Portal UI Tour
- Top-Level Concepts
- Linked Services
- Datasets

WEEK 2

Pipelines & Activities

- Pipeline Creation
- Copy Activity Deep Dive
- Parameters & Variables
- Lookup & Metadata
- Execute Pipeline
- Fail Activity

WEEK 3

Control Flow & Triggers

- ForEach / If / Switch / Until
- Schedule Trigger
- Tumbling Window
- Storage Event Trigger
- Trigger Parameterization
- Labs: 3 trigger types

WEEK 4

Integration Runtimes

- Azure AutoResolve IR
- Managed VNet IR
- Self-Hosted IR
- Linked Self-Hosted IR
- Hybrid Cloud Pipeline
- IR Monitoring

WEEK 5

Security & DevOps

- Global Parameters
- Managed Identity
- Key Vault Integration
- Private Endpoints
- ARM Templates
- Git / CI-CD

WEEK 6

Monitoring & Alerts

- Monitor Tab Deep Dive
- Pipeline Run Analysis
- Azure Monitor Alerts
- Logic Apps Notifications
- Log Analytics KQL
- Observability Dashboard

WEEK 7

Data Flows — Basics

- What is a Data Flow
- Mapping Data Flow
- Debug Mode
- Filter · Aggregate · Join
- Derived Column
- Conditional Split

WEEK 8

Data Flows — Advanced

- Union · Lookup · Sort
- Pivot · Unpivot · Flatten
- Flowlet · Schema Drift
- Deduplication Patterns
- SCD Type 1 & Type 2
- Capstone Project

CAPSTONE PROJECT

Build a complete Data Warehouse ETL with Incremental Loads, SCD Type 2, Monitoring & CI/CD deployment — using real Azure resources end-to-end



**Dedicated
Lab**

24/7 Access

**High
Quality**

Study
Materials

**Lifetime
Access**

to
Recordings

Hands-on
Assignments

**Doubts
Mentor**
for
Clarification

**Real-
World**
Projects

Community
Support



Job Roles — Azure Data Engineer

Roles you'll be qualified for after completing this course

Azure Data Engineer

ADF

ADLS

Pipelines

ETL Developer

Copy Activity

Data Flows

SQL

Data Integration Engineer

Linked Services

Connectors

SCD

Cloud Data Engineer

Azure

Orchestration

ARM

Data Pipeline Developer

Triggers

Scheduling

Git

BI Data Engineer

Transformations

Mapping DF

Reporting

Azure Solutions Architect

IR Design

Security

VNet

Data Platform Engineer

Monitoring

Alerts

DevOps

MODULE 1 — AZURE DATA FACTORY

1

Azure Data Factory (ADF)

Cloud-based ETL/ELT orchestration service — the backbone of Azure data pipelines

1.1 What is Azure Data Factory?

4 subtopics

[Introduction, use cases, and ADF architecture overview](#)

1.1.1 Definition & Purpose

Managed cloud **ETL/ELT service** for complex hybrid data integration. Orchestrates data movement and transformation across **90+ cloud and on-premises sources** at enterprise scale without managing any infrastructure.

HANDS-ON LABS

- Lab 1** Create your first Azure Data Factory resource in the Azure Portal from scratch
- Lab 2** Navigate the ADF Studio — explore Author, Monitor, and Manage tabs
- Lab 3** Identify the difference between ETL and ELT using a real dataset example in ADF
- Lab 4** Explore the built-in ADF templates gallery and deploy a sample copy pipeline
- Lab 5** Review the ADF pricing calculator — estimate cost for a simple daily copy pipeline

1.1.2 ADF Architecture

Control Plane (Azure-hosted UI & APIs for authoring/monitoring) + **Data Plane** (Integration Runtimes executing data movement). Actual data never passes through the Control Plane — it moves directly between source and sink via the IR.

HANDS-ON LABS

- Lab 1** Draw and label the ADF architecture diagram — Control Plane vs Data Plane
- Lab 2** Identify which ADF components live in the Control Plane vs Data Plane in the portal
- Lab 3** Create an Azure AutoResolve IR and observe it in the Manage tab
- Lab 4** Run a simple pipeline and trace which IR was used in the Monitor activity run details
- Lab 5** Compare the activity run input/output JSON to confirm data did not pass through ADF control plane

1.1.3 ADF Portal UI

Three main tabs: **Author** (create pipelines, datasets, linked services) · **Monitor** (track all pipeline, activity, and trigger runs) · **Manage** (linked services, IRs, triggers, global parameters, credentials).

HANDS-ON LABS

- Lab 1** Explore the Author tab — identify where pipelines, datasets, and data flows are created
- Lab 2** Use the Monitor tab to view pipeline runs and filter by status (Succeeded/Failed)
- Lab 3** Explore the Manage tab — list all linked services and integration runtimes
- Lab 4** Use the ADF search bar to find a specific pipeline or dataset by name
- Lab 5** Enable and explore the ADF diagnostic settings and link to a Log Analytics workspace

1.1.4 Top-Level Concepts Overview

Seven core building blocks: **Pipelines** · **Activities** · **Linked Services** · **Datasets** · **Triggers** · **Data Flows** · **Integration Runtimes**. Understanding how these relate to each other is the foundation of every ADF solution.

HANDS-ON LABS

- Lab 1** Map all 7 ADF concepts on a whiteboard and draw the dependency arrows between them
- Lab 2** Create one of each: a Linked Service, a Dataset, and a Pipeline — connect them end to end
- Lab 3** Add a Schedule Trigger to your pipeline and verify it appears in the Manage tab
- Lab 4** Create a simple Mapping Data Flow and attach it to a pipeline as an activity
- Lab 5** Review a published ADF ARM template JSON and identify each of the 7 concepts in the file

Migration

ADF → Microsoft Fabric Migration Assistant (Public Preview)

Built-in migration (March 2026) — assess, plan, and migrate ADF pipelines to Fabric Data Factory for free. Pipelines categorized as: Ready / Needs Review / Coming Soon / Unsupported.

March 2026 · Public Preview

Strategic

Microsoft shifting new features to Fabric Data Factory

New capabilities being built in Fabric Data Factory only. ADF remains fully supported — learning ADF transfers directly to Fabric (Track 4).

2025-2026 Roadmap

GA

Pipeline activity limit raised to 80 activities

Maximum activities per pipeline increased from 40 to 80, enabling more complex single-pipeline orchestrations.

April 2024

Sample Project — ADF Foundations Setup

Section 1.1 · End-to-end project tying all subtopic labs together

PROJECT

Build a fully configured ADF environment from scratch that copies a CSV file from Azure Blob Storage to Azure SQL Database, demonstrating all 7 ADF concepts in action.

Step 1 Create an Azure Data Factory resource, a Blob Storage account, and an Azure SQL Database

Step 2 Create Linked Services for Blob Storage (Account Key) and SQL Database (SQL Auth)

Step 3 Create source Dataset (CSV on Blob) and sink Dataset (SQL Table)

Step 4 Build a Pipeline with a Copy Activity connecting source to sink, run via Debug

Step 5 Publish and verify the pipeline run in the Monitor tab — confirm rows loaded in SQL

1.2 Pipelines

Creating, managing, and parameterizing ADF pipelines end to end

6 subtopics

1.2.1 What is a Pipeline

Logical grouping of activities that together perform a data engineering task. The **unit of execution** in ADF — you trigger, monitor, rerun, and manage at the pipeline level.

HANDS-ON LABS

- Lab 1** Create a new empty pipeline and rename it using a meaningful naming convention
- Lab 2** Add a Wait activity to the pipeline and run it via Debug — observe the run in Monitor
- Lab 3** Create two pipelines and link them using Execute Pipeline activity (synchronous mode)
- Lab 4** Rerun a failed pipeline from the Monitor tab and observe the rerun behavior
- Lab 5** View the pipeline JSON definition in Code view and identify each activity and its properties

1.2.2 Creation of Pipeline

Using the visual canvas — drag-and-drop activities and connect them with **success, failure, or completion** dependency arrows. Each arrow type controls which path executes next.

HANDS-ON LABS

- Lab 1** Drag a Copy Activity onto the canvas and connect it to a Wait activity with a Success dependency
- Lab 2** Add a Fail activity connected via a Failure dependency to the Copy Activity
- Lab 3** Create a pipeline with a Completion dependency — verify both branches trigger regardless of status
- Lab 4** Build a 3-activity sequential pipeline: Lookup → Copy → Stored Procedure
- Lab 5** Use the canvas Zoom and Auto Layout features to organize a complex multi-activity pipeline

1.2.3 Directory of Pipelines

Organizing pipelines in folders with consistent naming conventions — by domain, source system, or medallion layer (bronze/silver/gold).

HANDS-ON LABS

- Lab 1** Create a folder structure: bronze/, silver/, gold/ and move pipelines into each folder
- Lab 2** Create 3 pipelines named PL_Ingest_Sales, PL_Transform_Sales, PL_Load_Sales following naming conventions
- Lab 3** Create a master pipeline that calls all 3 child pipelines in sequence using Execute Pipeline
- Lab 4** Use the search bar to find a pipeline by partial name — practice navigating large factories
- Lab 5** Export the ARM template and verify the folder structure is preserved in the JSON output

1.2.4 Save, Debug & Trigger Now

Draft vs Published state — changes exist as drafts until you publish. **Debug mode**: run pipeline interactively with breakpoints. **Trigger Now**: ad-hoc run without a scheduled trigger.

HANDS-ON LABS

- Lab 1** Make a change to a pipeline — observe the unsaved indicator and save as draft without publishing
- Lab 2** Run a 3-activity pipeline in Debug mode — set a breakpoint on the middle activity and inspect its output
- Lab 3** Use Trigger Now on a published pipeline and pass a parameter value at runtime
- Lab 4** Intentionally fail an activity and debug the pipeline to observe the failure path in action
- Lab 5** Compare Debug run vs Trigger Now run in Monitor — identify differences in run type display

1.2.5 Publish Pipelines

Publish pushes draft changes to the live service. In **Git mode**: publish from collaboration branch after PR review. In **live mode**: publish deploys immediately. ARM template generated at publish time.

HANDS-ON LABS

- Lab 1** Publish a pipeline and download the generated ARM template JSON from the Publish panel
- Lab 2** Connect ADF to a GitHub repository and commit a pipeline change via a feature branch
- Lab 3** Merge a PR into the collaboration branch and publish to the live ADF service from Git mode
- Lab 4** Import an ARM template into a new ADF instance and verify the pipeline is recreated correctly
- Lab 5** Roll back to a previous published version using Git history — verify the pipeline reverts correctly

1.2.6 Parameters & Pipeline Parameters

Parameters: passed at trigger/execution time — immutable during run. **Variables**: set internally during run — mutable. Parameters enable the same pipeline to handle different files or tables without duplication.

HANDS-ON LABS

- Lab 1** Add a string parameter p_filename to a pipeline and pass it to a Copy Activity source path using @pipeline().parameters.p_filename
- Lab 2** Add a pipeline variable v_rowcount and set it using Set Variable after a Copy Activity output
- Lab 3** Run the same pipeline three times with three different parameter values — observe three separate source files copied
- Lab 4** Pass a parameter from a parent Execute Pipeline activity to a child pipeline parameter
- Lab 5** Use an Append Variable to build an array of processed filenames and log it using a Set Variable at the end

Sample Project — Dynamic Parameterized Pipeline

Section 1.2 · Build a reusable pipeline that processes any file dynamically

PROJECT

Build a single reusable pipeline that accepts a filename, table name, and environment as parameters, processes any CSV file from any container, and loads it into the correct SQL table — without duplicating pipelines per file.

- Step 1** Create a pipeline with parameters: **p_container, p_filename, p_tablename, p_environment**
- Step 2** Build a Copy Activity that uses all 4 parameters in source path and sink table name expressions
- Step 3** Create a master pipeline that calls this child pipeline 3 times with different filenames
- Step 4** Add a variable v_log array and append each filename after successful copy using Append Variable
- Step 5** Run via Trigger Now, pass parameters, verify all 3 files loaded, publish and commit to Git

1.3 Linked Services (LS)

3 subtopics

Connecting ADF to external data sources and compute resources

1.3.1 What is a Linked Service

Connection string + authentication info for an external resource. The "**data source connection**" of ADF. Every dataset and some activities reference a linked service — it is the bridge from ADF to the outside world.

HANDS-ON LABS

- Lab 1** Create a Linked Service for Azure Blob Storage using Account Key authentication
- Lab 2** Create a Linked Service for Azure SQL Database using SQL Authentication — run Test Connection
- Lab 3** Create a Linked Service for ADLS Gen2 using Managed Identity — verify access via IAM roles
- Lab 4** Intentionally enter a wrong connection string — observe and interpret the Test Connection error
- Lab 5** View the linked service JSON definition in Code view and identify the authenticationType property

1.3.2 Creating LS for Blob, ADLS, SQL Server — with Quick Test

Hands-on creation with **Test Connection (Quick Test)**. Authentication options:

Account Key — simple; dev/test only

SAS Token — time-limited delegated access

Managed Identity — most secure; production recommended

Service Principal — when Managed Identity unavailable

HANDS-ON LABS

Lab 1 Create a Blob LS using Account Key and successfully run Quick Test

Lab 2 Generate a SAS token for a Blob container and create a LS using SAS URI authentication

Lab 3 Assign Storage Blob Data Contributor role to ADF Managed Identity and create an ADLS LS using Managed Identity

Lab 4 Register an App Registration in Entra ID, create a Service Principal, and use it in an ADF Linked Service

Lab 5 Create an SQL Server Linked Service using SQL Auth — test with both correct and incorrect credentials to compare error messages

1.3.3 Parameterization of Linked Services

Making LS **dynamic using parameters** — dynamic server name, database name, container. Enables one LS to serve multiple environments (dev/test/prod) by injecting different values at runtime.

HANDS-ON LABS

- Lab 1** Add a parameter `p_serverName` to an SQL Linked Service and reference it in the Server field
- Lab 2** Add a parameter `p_databaseName` to the same LS — test connecting to two different databases using parameter values
- Lab 3** Create one parameterized Blob LS and use it to connect to dev-container and prod-container from the same pipeline
- Lab 4** Pass Linked Service parameter values from a Global Parameter so dev/prod environments switch automatically
- Lab 5** Use a Lookup activity to read environment config from a table and pass the result to a parameterized LS at runtime

Sample Project — Multi-Environment Connection Manager

Section 1.3 · One Linked Service that connects to dev, test, and prod dynamically

PROJECT

Build a single parameterized Linked Service architecture that allows the same pipeline to connect to dev, test, and prod SQL databases simply by changing a Global Parameter value — no duplicate linked services.

- Step 1** Create three Azure SQL DBs: **adf-dev, adf-test, adf-prod** with the same schema
- Step 2** Create one parameterized SQL Linked Service with `p_server` and `p_database` parameters
- Step 3** Create Global Parameters `gp_server` and `gp_database` — set them to dev values
- Step 4** Wire the LS parameters to the Global Parameters — run pipeline and verify data loads to dev DB
- Step 5** Change Global Parameters to prod values, rerun — verify data loads to prod DB with zero code changes

1.4 Datasets (DS)

3 subtopics

Defining data structures — files, tables, and parameterization

1.4.1 What is a Dataset

A **named view of data** pointing to where the data lives. References a Linked Service and adds schema, file path, or table name on top of it. Used by activities as source or sink targets.

HANDS-ON LABS

- Lab 1** Create a DelimitedText (CSV) Dataset pointing to a file in Blob Storage — import schema
- Lab 2** Create a Parquet Dataset on ADLS Gen2 — define the folder path and enable schema import
- Lab 3** Create an Azure SQL Table Dataset — browse and select an existing table from the connected DB
- Lab 4** Create a Query-based SQL Dataset that returns only rows where status = 'Active'
- Lab 5** Use the Preview Data button on a Dataset to verify it returns the expected rows and columns

1.4.2 Creating Datasets — For Files & For Tables in DB

For Files (Blob/ADLS): CSV, JSON, Parquet, Avro, ORC, Delta formats; schema import; define file or folder path.

For Tables in DB (SQL Server/Azure SQL): browse and select table or use a query-based dataset for custom SELECT.

HANDS-ON LABS

- Lab 1** Create datasets for each format: CSV, JSON, Parquet, Avro — point each to a different file in Blob
- Lab 2** Create a Delta format Dataset on ADLS Gen2 pointing to an existing Delta table folder
- Lab 3** Create an SQL Table Dataset using table browse — compare with a Query Dataset using a JOIN query
- Lab 4** Use a Copy Activity to copy from a CSV Dataset to a Parquet Dataset — verify format conversion
- Lab 5** Create a Binary Dataset for copying files without parsing content (e.g., images, PDFs)

1.4.3 How to Parameterize Datasets

Dynamic file paths, folder names, table names using **@dataset().paramName** syntax. One dataset definition handles different files or tables per pipeline run.

HANDS-ON LABS

- Lab 1** Add parameter `p_filename` to a CSV Dataset — reference it in the file path as `@dataset().p_filename`
- Lab 2** Add parameter `p_folderPath` — use it to dynamically switch between bronze/ and silver/ containers
- Lab 3** Add parameter `p_tableName` to an SQL Dataset — use it to dynamically target different tables
- Lab 4** Pass dataset parameters from a pipeline ForEach loop — process 5 different files using one parameterized dataset
- Lab 5** Combine parameterized LS + parameterized DS in one pipeline — fully dynamic source and sink

Sample Project — Generic File Ingestion Framework

PROJECT

Section 1.4 · One parameterized dataset that ingests any CSV file dynamically

Build a generic ingestion framework using a single parameterized source dataset and a single parameterized sink dataset that can ingest any CSV file from any folder into any SQL table — driven by a config table.

- Step 1** Create a SQL config table: **SourceContainer, SourceFile, SinkTable** with 5 rows
- Step 2** Create parameterized CSV source Dataset (`p_container`, `p_filename`) and SQL sink Dataset (`p_tablename`)
- Step 3** Use Lookup to read the config table, then ForEach to iterate over all 5 rows
- Step 4** Inside ForEach, use Copy Activity passing config values to dataset parameters
- Step 5** Run the pipeline — verify all 5 files are loaded into their respective SQL tables

1.5.1 Wait Activity

Pauses pipeline execution for a **specified duration in seconds**. Useful for throttling API calls, scheduling gaps, or waiting for a downstream system to become ready.

HANDS-ON LABS

- Lab 1** Add a Wait activity (30 seconds) between two Copy activities — observe the pause in Monitor
- Lab 2** Use a pipeline parameter to dynamically set the wait duration in seconds
- Lab 3** Build a retry pattern: Copy → Wait (60s) → Lookup to check if data arrived → conditional retry
- Lab 4** Use Wait to throttle an API call pipeline — add a 5-second wait between each ForEach iteration
- Lab 5** Chain Copy → Wait → Delete to simulate: copy a file, wait 10s, then delete the source

1.5.2 Variables — Set / Get / Append Variable

Pipeline-scoped mutable variables. **Set Variable**: assign a value. **Append Variable**: add to array. **Get Variable**: read via `@variables('name')`.

HANDS-ON LABS

- Lab 1** Create a string variable `v_status` — set it to "Started", then "Completed" and log both values
- Lab 2** Create an array variable `v_files` — use Append Variable in a ForEach to collect all processed filenames
- Lab 3** Use Set Variable to capture a Copy Activity output `rowsCopied` value — store in `v_rowcount`
- Lab 4** Use an If Condition to check if `v_rowcount > 0` — route to success or empty-file Fail path
- Lab 5** Build an audit log pipeline: use variables to track pipeline name, start time, end time, row count and insert to SQL audit table

1.5.3 Copy Activity — Deep Dive

Core activity for data movement between **90+ connectors**. Six tabs: **Source** · **Sink** · **General** · **Mapping** · **Settings** · **User Properties**.

HANDS-ON LABS

- Lab 1** Copy a CSV from Blob to SQL — manually map columns in the Mapping tab, rename one column
- Lab 2** Configure the Sink with a pre-copy script: TRUNCATE TABLE before each load
- Lab 3** Set General tab: timeout 1 hour, retry 3, retry interval 30s — intentionally fail and observe retries
- Lab 4** Enable Fault Tolerance in Settings — skip incompatible rows and log them to a separate Blob file
- Lab 5** Add User Properties (SourceSystem=SAP, Layer=Bronze) — filter pipeline runs by these properties in Monitor
- Lab 6** Copy Blob → ADLS Gen2 as Parquet using the Copy Activity format conversion — verify file size reduction
- Lab 7** Use wildcard file path in Source to copy all *.csv files from a folder in one Copy Activity run

1.5.4 Lookup Activity

Reads data from a dataset and returns it as activity output via **@activity('Lookup').output**. Used to fetch config values, get a dynamic file list, or check record existence.

HANDS-ON LABS

- Lab 1** Use Lookup to read a watermark value (last_load_date) from a SQL config table
- Lab 2** Pass the Lookup output (watermark date) to a Copy Activity source query as a parameter
- Lab 3** Use Lookup to return a list of table names — pass to ForEach to process each table dynamically
- Lab 4** Use Lookup with First Row Only disabled — return all rows and access via .value array in ForEach
- Lab 5** Build a Lookup → If Condition pattern: check if a record exists — insert only if not found

1.5.5 Stored Procedure, Get Metadata, Delete, Execute Pipeline, Fail

Stored Procedure: call SQL SP for post-load transforms. **Get Metadata:** file/folder metadata. **Delete:** remove source files. **Execute Pipeline:** call child pipelines. **Fail:** enforce business rules with custom errors.

HANDS-ON LABS

- Lab 1** Create a SQL stored procedure that updates a watermark table — call it via Stored Procedure activity after a Copy
- Lab 2** Use Get Metadata to check if a file exists in Blob — use childItems output with If Condition to proceed or skip
- Lab 3** Build: Copy file → verify rowsCopied > 0 → Delete source file. Use Delete activity on the Failure path to clean up partial loads
- Lab 4** Create a child pipeline — call it from a parent using Execute Pipeline (synchronous) and verify run order in Monitor
- Lab 5** Add a Fail activity: check rowsCopied = 0 → Fail with message "Source file is empty" and error code 400 — verify alert triggers

1.5.6 Azure Function Activity

Invokes an Azure Function app for lightweight custom compute logic — API calls, custom validations, sending notifications, or any logic not available natively in ADF.

HANDS-ON LABS

- Lab 1** Create an Azure Function (HTTP trigger) that returns a JSON response — call it from ADF using Azure Function activity
- Lab 2** Pass a pipeline parameter as the function request body — read and return it in the Function response
- Lab 3** Use an Azure Function to validate a file checksum — fail the ADF pipeline if validation fails
- Lab 4** Create a Function that sends an email via SendGrid — call it from ADF on pipeline success
- Lab 5** Access the Function activity output in a downstream Set Variable activity to capture the Function return value

1.5.7 Iterations & Conditionals

Control flow: **Filter** (filter array), **If Condition** (true/false branch), **For Each** (iterate array), **Switch** (multi-branch), **Until** (loop until condition).

HANDS-ON LABS

- Lab 1** Use Filter activity on a Lookup output array — keep only rows where status = 'Pending'
- Lab 2** Use If Condition to check if rowsCopied > 0 — route success to Stored Procedure, failure to Fail activity
- Lab 3** Use ForEach (sequential mode) to iterate 5 table names from Lookup and copy each to ADLS
- Lab 4** Use Switch to route pipeline execution to different Copy activities based on a p_environment parameter (dev/test/prod)
- Lab 5** Use Until loop: copy a file, wait 30s, check if a flag file exists — keep looping until the flag file appears (max 5 iterations)

Sample Project — Incremental Load Pipeline with Watermark

Section 1.5 · Real-world incremental ETL pattern using 6 activity types

PROJECT

Build a production-grade incremental load pipeline that reads only new/changed records from a SQL source since the last run, copies them to ADLS Gen2 as Parquet, and updates the watermark — the most common real-world ADF pattern.

- Step 1 Lookup:** read last_load_datetime watermark from a SQL config table
- Step 2 Copy Activity:** source query = `SELECT * FROM orders WHERE modified_date > '@{activity('Lookup').output.firstRow.watermark}'`
- Step 3 Set Variable:** capture rowsCopied from Copy output into v_rowcount
- Step 4 If Condition:** if v_rowcount > 0 → update watermark via Stored Procedure; else → Fail with "No new data"
- Step 5** Schedule with Tumbling Window Trigger — run daily, verify incremental rows only load each time

1.6 Triggers

5 subtopics

Scheduling and event-based pipeline execution — all trigger types

1.6.1 What is a Trigger

Defines when a pipeline execution should kick off. Can be **time-based** (wall clock) or **event-based** (file arrival). One trigger can be associated with multiple pipelines.

HANDS-ON LABS

- Lab 1** View all trigger types available in ADF Manage tab — list Schedule, Tumbling Window, Storage Event
- Lab 2** Create a trigger and associate it with two different pipelines — verify both trigger on schedule
- Lab 3** Start and stop a trigger from the Manage tab — observe pipeline run behavior in Monitor
- Lab 4** View trigger run history in Monitor — filter by trigger name and date range
- Lab 5** Export a trigger as ARM template JSON — identify startTime, endTime, recurrence frequency in the JSON

1.6.2 Schedule Trigger

Runs pipeline at a fixed wall-clock time using recurrence (hourly, daily, cron). **Use when:** regular batch loads at predictable, fixed times.

HANDS-ON LABS

- Lab 1** Create a Schedule Trigger that fires daily at 2:00 AM UTC — associate it with a Copy pipeline
- Lab 2** Create a Schedule Trigger with hourly recurrence — observe runs every hour in Monitor
- Lab 3** Create a Schedule Trigger with a custom cron expression — e.g., every Monday and Friday at 6 AM
- Lab 4** Set a start date and end date on a Schedule Trigger — verify it stops firing after the end date
- Lab 5** Pass @trigger().scheduledTime to a pipeline parameter — log the trigger time in a SQL audit table

1.6.3 Tumbling Window Trigger

Fixed-size non-overlapping sequential windows. Maintains state; supports retry and window dependency. **Use when:** sequential time-windowed processing with backfill capability.

HANDS-ON LABS

- Lab 1** Create a Tumbling Window Trigger with 1-hour window size — observe windowStartTime and windowEndTime in Monitor
- Lab 2** Pass @trigger().outputs.windowStartTime to a Copy Activity source query as the incremental load filter
- Lab 3** Set a past startTime on a Tumbling Window Trigger — observe backfill windows being created and run sequentially
- Lab 4** Configure a retry policy on Tumbling Window Trigger — intentionally fail and verify automatic retry
- Lab 5** Set up a Tumbling Window trigger dependency — Trigger B only runs after Trigger A's window succeeds

1.6.4 Storage Event Trigger

Fires when a blob is **created or deleted** in Azure Storage. Supports path prefix/suffix filters. **Use when:** event-driven pipelines that should process a file immediately on arrival.

HANDS-ON LABS

- Lab 1** Create a Storage Event Trigger on blob created — upload a file and verify the pipeline fires immediately
- Lab 2** Add a blob path suffix filter (.csv) — verify trigger fires only for CSV files, not other types
- Lab 3** Add a blob path prefix filter (landing/) — verify trigger fires only for files in landing/ folder
- Lab 4** Pass @trigger().outputs.body.fileName to a pipeline parameter — use it to target the exact arrived file in Copy Activity
- Lab 5** Create a blob delete trigger — fire it by deleting a file and verify the pipeline runs a cleanup logic

1.6.5 Trigger with Parameterization

Pass trigger-specific values to pipeline parameters using **@trigger() system variables**: `scheduledTime`, `startTime`, `windowStartTime`, `windowEndTime`, `body.fileName`.

HANDS-ON LABS

- Lab 1** Pass `@trigger().scheduledTime` to a pipeline parameter `p_runDate` — log it in a SQL audit table
- Lab 2** Use `@trigger().outputs.windowStartTime` as a filter start and `windowEndTime` as filter end in a Copy source query
- Lab 3** Use `@trigger().outputs.body.fileName` from a Storage Event Trigger to dynamically name the output file in ADLS
- Lab 4** Combine `@trigger().startTime` and a pipeline variable to build a dynamic output folder path (year/month/day)
- Lab 5** Pass trigger name `@trigger().name` to a pipeline parameter — use it to insert into a `run_log` table for auditability

Sample Project — Event-Driven File Processing Pipeline

Section 1.6 · Auto-process any CSV file the moment it lands in storage

PROJECT

Build a fully automated event-driven pipeline that detects when a CSV file lands in a Blob landing zone, automatically copies it to ADLS Gen2 as Parquet, logs the run to SQL, and deletes the source file.

- Step 1** Create a Storage Event Trigger on blob created in `landing/` container — filter suffix `.csv`
- Step 2** Pass `@trigger().outputs.body.fileName` and `folderPath` to pipeline parameters
- Step 3** Copy Activity: CSV source (dynamic path from trigger) → ADLS Gen2 Parquet sink (dated folder)
- Step 4** Stored Procedure: insert `run_log` record (filename, rows_copied, trigger_time, status)
- Step 5** Delete Activity: remove source CSV from `landing/` — upload 3 test files and verify all 3 are auto-processed

1.7 Integration Runtimes (IR)

4 subtopics

The compute infrastructure that executes ADF data movement and transformations

1.7.1 Azure AutoResolve IR

Default, fully managed Azure IR. Auto-selects region closest to data. Serverless — no provisioning needed. **Use when:** cloud-to-cloud movement between Azure services with no VNet isolation required.

HANDS-ON LABS

- Lab 1** Run a Blob to SQL Copy pipeline — verify AutoResolveIntegrationRuntime is used in the activity run details
- Lab 2** Create a named Azure IR (not AutoResolve) in East US — assign it to a Copy Activity and observe region difference
- Lab 3** Compare copy performance: AutoResolve IR vs a fixed-region Azure IR — same source/sink, note timing differences
- Lab 4** Assign AutoResolve IR to a Mapping Data Flow activity — verify Spark cluster provisioning in Monitor
- Lab 5** View IR usage metrics in Azure Monitor — identify which IR type ran for each pipeline execution

1.7.2 Azure Managed Virtual Network IR

Azure IR deployed inside a managed VNet. Enables private endpoint connectivity. **Use when:** connecting to resources behind private endpoints without exposing data to the internet.

HANDS-ON LABS

- Lab 1** Enable Managed Virtual Network on an Azure IR in the Manage tab — note the approval delay
- Lab 2** Create a Managed Private Endpoint for an ADLS Gen2 account — approve the private endpoint request
- Lab 3** Assign the Managed VNet IR to a Linked Service — run a Copy pipeline and verify traffic uses the private endpoint
- Lab 4** Create a Managed Private Endpoint for Azure SQL DB — run a Copy pipeline using private connectivity
- Lab 5** Compare pipeline run time: AutoResolve IR vs Managed VNet IR — same copy job, different IRs

1.7.3 Self-Hosted IR

IR agent on an on-premises machine or private VM. Bridges on-prem data sources to ADF.

Use when: connecting to on-prem SQL Server, Oracle, SAP, or private file shares.

HANDS-ON LABS

- Lab 1** Create a Self-Hosted IR in ADF — download and install the agent on a Windows VM
- Lab 2** Register the Self-Hosted IR using the authentication key — verify status shows Running in Manage tab
- Lab 3** Create a SQL Server Linked Service on the VM — assign Self-Hosted IR — run Test Connection
- Lab 4** Copy data from on-prem SQL Server (via Self-Hosted IR) to ADLS Gen2 (via Azure IR) — hybrid pipeline
- Lab 5** Monitor Self-Hosted IR health — view node status, CPU/memory usage in the IR monitoring page

1.7.4 Linked Self-Hosted IR

Share an existing Self-Hosted IR **across multiple ADF instances** without installing more agents. **Use when:** enterprise with multiple factories needing on-prem connectivity.

HANDS-ON LABS

- Lab 1** Create a second ADF instance — set up a Linked Self-Hosted IR referencing the existing shared IR
- Lab 2** Create a Linked Service in the second ADF using the Linked Self-Hosted IR — run Test Connection
- Lab 3** Run a pipeline from the second ADF that copies from on-prem SQL via the shared Linked IR
- Lab 4** Monitor which ADF factory is using the shared IR from the IR host machine's event log
- Lab 5** Revoke access to Linked IR from the second ADF — verify the Linked Service Test Connection now fails

Sample Project — Hybrid Cloud Pipeline (On-prem to Azure)

Section 1.7 · Copy from on-premises SQL Server to ADLS Gen2 via Self-Hosted IR

PROJECT

Build a hybrid pipeline that uses Self-Hosted IR to connect to an on-premises SQL Server database, incrementally extract new orders, and load them into ADLS Gen2 as Parquet — simulating a real enterprise migration scenario.

Step 1 Install Self-Hosted IR on a local/VM SQL Server machine and register it with ADF

Step 2 Create Linked Services: SQL Server (Self-Hosted IR) and ADLS Gen2 (Azure IR)

Step 3 Lookup watermark → Copy incremental rows from on-prem SQL → ADLS Gen2 as Parquet

Step 4 Update watermark in on-prem SQL config table via Stored Procedure through Self-Hosted IR

Step 5 Schedule with Tumbling Window Trigger — run for 5 consecutive windows and verify incremental loads

1.8 Source Control & ARM Templates

2 subtopics

[Version control for ADF using Git and ARM template export/import](#)

1.8.1 ARM Template Export / Import

Export entire ADF as an **Azure Resource Manager JSON template**. Import into another factory for deployment or promotion. Enables **laC (Infrastructure as Code)** for ADF.

HANDS-ON LABS

- Lab 1** Export an ADF with 3 pipelines as ARM template — inspect the JSON and identify each pipeline
- Lab 2** Import the ARM template into a new ADF instance — verify all pipelines and linked services are recreated
- Lab 3** Edit the ARM template JSON to change a connection string value — re-import and verify the change
- Lab 4** Use Azure DevOps Pipelines to deploy an ADF ARM template to a target environment automatically
- Lab 5** Create ARM template parameter files for dev, test, and prod — deploy the same template to 3 environments

1.8.2 Git Integration — Azure DevOps / GitHub

Connect ADF to Azure DevOps Git or GitHub. Feature branches, pull requests, publish from collaboration branch. Every change is tracked with full diff, rollback, and audit trail.

HANDS-ON LABS

- Lab 1** Connect ADF to a GitHub repository — configure collaboration branch as 'main'
- Lab 2** Create a feature branch, add a new pipeline, commit, and raise a PR to merge into main
- Lab 3** Approve and merge the PR — publish to ADF live service and verify the new pipeline is live
- Lab 4** View Git commit history — roll back a pipeline to a previous version using Git revert
- Lab 5** Set up branch policies in Azure DevOps to require at least one PR reviewer before merge to main

Sample Project — CI/CD Pipeline for ADF Deployment

Section 1.8 · Automate ADF deployment from dev to prod using Git + Azure DevOps

PROJECT

Set up a complete CI/CD workflow: ADF in Git mode → PR to main → Azure DevOps Pipeline auto-deploys ARM template to prod ADF using environment-specific parameter files.

- Step 1** Connect dev-ADF to Azure DevOps Git — set collaboration branch as 'adf_publish'
- Step 2** Create ARM parameter files: ARMTemplateParametersForFactory_dev.json and _prod.json
- Step 3** Create an Azure DevOps Release Pipeline triggered by publish to adf_publish branch
- Step 4** Add ARM deployment step using prod parameter file — deploy to prod-ADF resource group
- Step 5** Make a pipeline change in dev, publish, verify prod auto-deploys within 5 minutes

1.9.1 Global Parameters

Factory-level constants accessible via `@pipeline().globalParameters.paramName`.

Use for: environment name, base storage path, shared config values. Can be overridden per environment at CI/CD deployment time.

HANDS-ON LABS

- Lab 1** Create Global Parameters: `gp_environment` (string), `gp_storageAccount` (string) — use them in a pipeline expression
- Lab 2** Reference `@pipeline().globalParameters.gp_storageAccount` in a Linked Service parameter
- Lab 3** Export ARM template — locate global parameters in the JSON and modify values for prod environment
- Lab 4** Override global parameters during ARM deployment using the parameter file — verify prod values are applied
- Lab 5** Use a global parameter `gp_emailRecipient` in a Logic App alert trigger — change recipient without touching pipelines

1.9.2 Credentials

Managed Identity and user-assigned managed identity credentials. ADF resolves identity at runtime from Azure Entra ID — no keys or passwords stored in ADF artifacts.

HANDS-ON LABS

- Lab 1** Find the System-Assigned Managed Identity of your ADF — copy its Object ID
- Lab 2** Assign Storage Blob Data Contributor role to ADF Managed Identity on an ADLS Gen2 account — create a Linked Service using MI
- Lab 3** Create a User-Assigned Managed Identity — assign it to ADF and create a Credential object in the Manage tab
- Lab 4** Use ADF Managed Identity to access an Azure Key Vault secret — create a Key Vault Linked Service using MI
- Lab 5** Remove the role assignment from ADF MI — verify the Linked Service Test Connection now fails with 403 Forbidden

1.9.3 Managed Private Endpoints

Private endpoints within ADF's managed VNet — connect to Azure resources privately over the Microsoft backbone. No traffic leaves the Azure network.

HANDS-ON LABS

- Lab 1** Enable Managed Virtual Network on a new Azure IR — verify it appears in the Manage tab
- Lab 2** Create a Managed Private Endpoint for ADLS Gen2 — approve the private endpoint in the storage account networking settings
- Lab 3** Create a Managed Private Endpoint for Azure SQL DB — run a Copy Activity using this private endpoint
- Lab 4** Disable public network access on the ADLS Gen2 account — verify pipeline still succeeds via private endpoint
- Lab 5** View the private endpoint connection in ADLS Gen2 networking settings — confirm provisioning state = Approved

Sample Project — Secure Enterprise ADF Setup

Section 1.9 · Zero-credential ADF factory using Managed Identity + Private Endpoints + Key Vault

PROJECT

Build a fully secure ADF factory where no passwords, keys, or connection strings exist anywhere in ADF — all authentication uses Managed Identity, all secrets are in Key Vault, and all connectivity uses private endpoints.

- Step 1** Create ADF with Managed VNet IR — create Managed Private Endpoints for ADLS, SQL DB, Key Vault
- Step 2** Grant ADF Managed Identity: Storage Blob Contributor (ADLS), DB Reader (SQL), Key Vault Secrets User
- Step 3** Store SQL password in Key Vault — create a Key Vault Linked Service using ADF Managed Identity
- Step 4** Create all Linked Services using Managed Identity (ADLS) and Key Vault reference (SQL) — no credentials stored in ADF
- Step 5** Run a Copy pipeline end-to-end — disable all public access on storage and SQL, verify pipeline still succeeds via private network

1.10 Monitoring ADF Jobs & Alerts

2 subtopics

[Tracking pipeline runs, debugging failures, failure notifications via Logic Apps](#)

1.10.1 Monitoring ADF Jobs

The **Monitor tab** shows Pipeline runs, Activity runs, and Trigger runs. Filter by status, date, pipeline name. **Rerun** from point of failure. Click any activity to view input/output JSON.

HANDS-ON LABS

- Lab 1** Run a pipeline with 4 activities — explore the Monitor tab and click each activity to view its input and output JSON
- Lab 2** Filter Monitor by Status=Failed for the past 7 days — identify the failed activities and their error messages
- Lab 3** Intentionally fail a Copy Activity — use Rerun from Monitor to restart only from the failed activity
- Lab 4** Use User Properties on activities to add custom tags — filter Monitor runs by those tags
- Lab 5** Enable ADF Diagnostic Logs — send to Log Analytics and query pipeline failures using KQL

1.10.2 Alerts — Failure Notifications using Logic Apps

Create **Azure Monitor alerts** on ADF pipeline failure events. Use **Logic Apps** as the action to send email, Teams, or Slack notifications. Config: alert rules → action groups → Logic App HTTP trigger.

HANDS-ON LABS

- Lab 1** Create an Azure Monitor Alert Rule on ADF PipelineRunsFailed metric — threshold = 1
- Lab 2** Create an Action Group with email notification — link it to the alert rule — intentionally fail a pipeline and verify email received
- Lab 3** Create a Logic App with HTTP trigger that sends a Teams message — call it from the Action Group
- Lab 4** Enrich the Teams alert message with pipeline name, run ID, and error message from the ADF Monitor API
- Lab 5** Create an alert for pipeline run duration > 30 minutes — verify an alert fires on a long-running pipeline

Sample Project — ADF Observability Dashboard

Section 1.10 · Full monitoring: alerts + Teams notifications + KQL Log Analytics queries

PROJECT

Build a complete ADF observability setup: pipeline failure alerts sent to Teams, a Log Analytics workspace collecting all ADF diagnostic logs, and a KQL query dashboard showing daily run success rates.

Step 1 Enable ADF Diagnostic Settings — send all logs and metrics to a Log Analytics Workspace

Step 2 Create an Azure Monitor Alert for PipelineRunsFailed — action group calls a Logic App HTTP trigger

Step 3 Logic App formats and sends a Teams adaptive card: pipeline name, error, run ID, timestamp

Step 4 Write KQL queries in Log Analytics: daily success rate, top 5 slowest pipelines, failure trend by day

Step 5 Pin KQL queries to an Azure Dashboard — share with team as a live ADF health dashboard

MODULE 2 — DATA FLOWS IN ADF

2

Data Flows in ADF

Visual, code-free data transformation engine powered by Apache Spark — all inside ADF, no Databricks needed

2.1 Introduction to Data Flows

3 subtopics

What Data Flows are, Mapping Data Flow, and Debug mode

2.1.1 What is a Data Flow

Visually designed transformation logic that **executes as Spark jobs** on a managed cluster. No coding required — ADF generates the underlying Spark code automatically.

HANDS-ON LABS

- Lab 1** Create your first Mapping Data Flow — add a CSV source from Blob and preview the data
- Lab 2** Inspect the auto-generated Spark script from a Data Flow using the Script view
- Lab 3** Add a Data Flow activity to a pipeline — run end-to-end and verify the output in the sink
- Lab 4** Review the Data Flow activity run in Monitor — identify Spark cluster startup time vs execution time
- Lab 5** Compare a simple CSV-to-SQL Copy Activity vs a Data Flow doing the same task — compare performance and flexibility

2.1.2 Mapping Data Flow

Primary Data Flow type. Build transformation logic by connecting **sources** → **transformations** → **sinks** on a visual canvas. Each transformation operates on the incoming stream.

HANDS-ON LABS

- Lab 1** Build a Mapping Data Flow: Blob CSV source → Derived Column (add FullName column) → SQL sink
- Lab 2** Add multiple transformations in sequence: Filter → Aggregate → Sort → Sink
- Lab 3** Use multiple sources in one Data Flow — join two CSV files on a common key
- Lab 4** Add multiple sinks to one Data Flow — route different records to different SQL tables using Conditional Split
- Lab 5** Add Data Flow parameters — pass them from the pipeline activity and use them in a source query filter

2.1.3 Data Flow Debug Mode

Interactive **Spark debug cluster** for testing row-by-row with live data previews. Stays warm for reuse. Important: **debug clusters incur cost while active** — turn off when done.

HANDS-ON LABS

- Lab 1** Enable Debug mode — observe the cluster starting up and note how long it takes to become Ready
- Lab 2** Click Data Preview on each transformation in sequence — observe how rows change at each step
- Lab 3** Use the expression builder with Debug active — test an `iif()` expression and preview the result live
- Lab 4** Set Debug row limit to 100 rows — preview a large dataset and verify only 100 rows are returned
- Lab 5** Turn off Debug mode — verify the cluster shuts down and the data preview panel is disabled

What's New in ADF Data Flows

• LIVE

Source: learn.microsoft.com/azure/data-factory/whats-new

GA Mapping Data Flows upgraded to Apache Spark 3.4

Upgraded from Spark 3.3 to 3.4 (August 2025). Better performance, Delta Lake support, enhanced Python/pandas compatibility, new SQL functions in expression builder.

August 2025

Upcoming Future enhancements moving to Fabric — Lakeflow Designer

New Data Flow features will be in Fabric's Lakeflow Designer (visual no-code canvas, GA July 2026). ADF Data Flow knowledge transfers directly — covered in Track 4.

2025-2026 Roadmap

Sample Project — Customer Data Cleansing Flow

Section 2.1 · Transform raw CSV data using a Mapping Data Flow end-to-end

PROJECT

Build a Data Flow that reads raw customer CSV from Blob, cleanses it (trim whitespace, fix phone formats, validate email), and writes cleaned data to Azure SQL DB — your first end-to-end Data Flow project.

Step 1 Create a Mapping Data Flow — add Blob CSV source with 1000 customer rows

Step 2 Add Derived Column: `trim(name)`, `upper(email)`, `regexReplace(phone, '^[0-9]', '')` for cleansing

Step 3 Add Filter: remove rows where email does not contain '@' using `iif(locate('@',email)>0, true(), false())`

Step 4 Add SQL sink — map cleaned columns — run in Debug and preview each transformation step

Step 5 Add to a pipeline — run end-to-end — verify cleaned rows loaded to SQL and invalid rows were filtered out

2.2.1 Filter

Removes rows not matching a condition. Like SQL **WHERE**. Use to exclude nulls, filter date ranges, or apply business-rule row filters.

HANDS-ON LABS

- Lab 1** Filter rows where status = 'Active' — preview before and after to verify row count drops
- Lab 2** Filter rows where sale_date >= 2024-01-01 using toDate() expression
- Lab 3** Filter out null values in a required column using !isNull(customer_id)
- Lab 4** Chain two Filter transformations — first filter by country, then by amount > 1000
- Lab 5** Use a Data Flow parameter in the Filter condition — pass the filter value from the pipeline at runtime

2.2.2 Aggregate

Groups data and computes aggregations: **SUM, COUNT, AVG, MAX, MIN, COLLECT**. Like SQL GROUP BY.

HANDS-ON LABS

- Lab 1** Group by region — compute SUM(sales_amount) and COUNT(order_id) per region
- Lab 2** Compute AVG(salary) and MAX(salary) grouped by department
- Lab 3** Use Aggregate without a group-by key — compute grand total SUM(revenue) across all rows
- Lab 4** Use collect() to create an array of all product names per customer — flatten it in a later Flatten transformation
- Lab 5** Group by month(order_date) and year(order_date) — compute monthly revenue and compare with a SQL query result

2.2.3 Join

Combines two streams. Types: **Inner** · **Left Outer** · **Right Outer** · **Full Outer** · **Cross** · **Exists**.

HANDS-ON LABS

- Lab 1** Inner Join orders and customers on customer_id — verify only matched rows appear in preview
- Lab 2** Left Outer Join orders and products on product_id — verify orders without products show null product columns
- Lab 3** Full Outer Join two employee datasets from different source systems — identify unmatched rows in both
- Lab 4** Join on multiple keys: customer_id AND order_date — verify compound join conditions work correctly
- Lab 5** Use Cross Join on a small dimension table — verify row count equals left_rows × right_rows

2.2.4 Conditional Split

Routes rows to different output streams based on conditions — like multi-way IF/CASE. First matching condition wins; unmatched rows go to default stream.

HANDS-ON LABS

- Lab 1** Split orders into 3 streams: High (amount>1000), Medium (amount 100-1000), Low (amount<100)
- Lab 2** Route rows by country code — UK to one SQL table, US to another, Others to a third
- Lab 3** Split valid vs invalid rows (null check) — load valid to target table, invalid to an error log file
- Lab 4** Use Conditional Split for SCD: New records to Insert stream, Changed records to Update stream
- Lab 5** Verify the default stream captures unmatched rows — confirm all rows are accounted for across all output streams

2.2.5 Derived Column

The **most versatile transformation** — adds new columns or updates existing ones using expressions. String manipulation, date formatting, type casting, conditional logic, null handling.

HANDS-ON LABS

- Lab 1** Add a FullName column: `concat(firstName, ' ', lastName)` using the expression builder
- Lab 2** Add LoadDate column: `currentDate()` — add LoadTimestamp: `currentTimestamp()`
- Lab 3** Cast string column to integer: `toInteger(order_amount)` — handle conversion errors with `iif(isNull(toInteger(x)), 0, toInteger(x))`
- Lab 4** Add a Category column using `iif()`: `iif(amount>1000,'Premium', iif(amount>100,'Standard','Basic'))`
- Lab 5** Use `regexReplace()` to clean a phone number column — remove all non-digit characters

2.2.6 Exists

Checks whether rows exist in another stream — like SQL **EXISTS** / **NOT EXISTS**. Key component in SCD Type 2 pattern to detect new vs changed records.

HANDS-ON LABS

- Lab 1** Use Exists to find new customers: source = CSV, lookup = SQL — keep rows NOT in SQL (new records)
- Lab 2** Use Exists to find changed records: match on ID, check if any other column differs using `hashAll()`
- Lab 3** Use Exists as a deduplication check — keep only rows from source that don't already exist in target
- Lab 4** Chain Exists → Conditional Split: `exist=true` → update stream; `exist=false` → insert stream
- Lab 5** Use NOT Exists to find orphan records — find orders with no matching `customer_id` in the customers table

2.2.7 Union

Combines rows from **multiple input streams** into one output — like SQL UNION ALL. Used to merge sources or re-combine after Conditional Split.

HANDS-ON LABS

- Lab 1** Union two CSV files with the same schema — verify combined row count = file1 rows + file2 rows
- Lab 2** Union 3 regional sales files (North, South, East) into a single output stream
- Lab 3** Add a source_region column in each stream using Derived Column before Union — tag each row's origin
- Lab 4** After a Conditional Split (new/changed/unchanged), Union only new and changed streams — load to sink
- Lab 5** Union By Name (not By Position) — verify column matching works when source schemas have different column orders

2.2.8 Lookup

Enriches the primary stream with data from a lookup dataset. Supports non-equi-join and fuzzy matching. Returns first match or all matches.

HANDS-ON LABS

- Lab 1** Lookup a country code table — enrich each order row with country_name from the lookup
- Lab 2** Use multiple match mode — verify each source row gets all matching lookup rows (one-to-many)
- Lab 3** Add a null handling Derived Column after Lookup — replace null lookup results with 'Unknown'
- Lab 4** Lookup a currency conversion table — multiply order_amount by the returned exchange_rate
- Lab 5** Use a parameterized Lookup source — pass the lookup table name from a pipeline parameter at runtime

2.2.9 Sort

Orders rows by one or more columns ascending or descending. Note: **expensive on large datasets** as it prevents parallelism — use only when the sink requires ordered data.

HANDS-ON LABS

- Lab 1** Sort orders by `order_date` descending — preview to verify newest orders appear first
- Lab 2** Sort by `customer_name` ascending, then by `amount` descending — verify compound sort order
- Lab 3** Sort by a Derived Column expression: `year(order_date), month(order_date)`
- Lab 4** Compare pipeline run time: Data Flow with Sort vs without Sort on 100K rows — observe performance impact
- Lab 5** Sort then Rank — use Sort to define order before applying `ROW_NUMBER` in a following Rank transformation

2.2.10 Group By / Window Functions

Applies **window functions by partition**: `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `LEAD`, `LAG` over ordered partitions — key for deduplication and SCD Type 2.

HANDS-ON LABS

- Lab 1** Add `ROW_NUMBER()` partitioned by `customer_id`, ordered by `order_date desc` — assign row numbers to each customer's orders
- Lab 2** Use `LEAD(amount, 1)` to get the next order amount for each customer row
- Lab 3** Compute running total: `cumSum(amount)` partitioned by `customer_id`, ordered by `order_date`
- Lab 4** Apply `DENSE_RANK()` to rank products by revenue within each category
- Lab 5** Chain Group By → Filter (`rn==1`) to deduplicate — keep only the most recent record per customer

2.2.11 Rank

Assigns rank values within partitions: **RANK**, **DENSE_RANK**, **ROW_NUMBER**. Used to pick the latest record version for deduplication and SCD Type 2.

HANDS-ON LABS

- Lab 1** Apply RANK() to score customers by total purchase amount within their region
- Lab 2** Compare RANK vs DENSE_RANK on a dataset with ties — observe gap vs no-gap behavior
- Lab 3** Use ROW_NUMBER() for deduplication — keep only rn=1 rows after ranking by latest modified date
- Lab 4** Apply Rank partitioned by product_category ordered by sales_amount — find top 3 products per category
- Lab 5** Use Rank + Filter (rank <= 3) to extract the top 3 customers per region from a large dataset

2.2.12 Unpivot

Converts **columns into rows** — wide to long format (inverse of Pivot). Useful when source has one column per month/category and you need a single normalized value column.

HANDS-ON LABS

- Lab 1** Unpivot a wide monthly sales table (Jan, Feb, Mar... columns) into Month, SalesAmount rows
- Lab 2** After Unpivot, add a Derived Column to convert month name to month number using locate()
- Lab 3** Unpivot a product attribute table (Color, Size, Weight as separate columns) into Attribute, Value rows
- Lab 4** Chain Unpivot → Filter to remove rows where the unpivoted value is null or zero
- Lab 5** Unpivot → Aggregate: after normalizing columns to rows, aggregate SUM by the new key column

2.2.13 Pivot

Converts **rows into columns** — long to wide format (like SQL PIVOT). Category values become separate columns with aggregated values.

HANDS-ON LABS

- Lab 1** Pivot a long sales table by region — create columns for North, South, East, West with SUM(amount)
- Lab 2** Pivot monthly data — group by product, pivot by month name, aggregate SUM(quantity)
- Lab 3** Use a parameterized pivot key list — pass dynamic column names from a pipeline parameter
- Lab 4** Add a Derived Column after Pivot to rename pivoted columns to friendly display names
- Lab 5** Combine Aggregate → Pivot: first aggregate raw rows by day, then pivot by category to create a crosstab

2.2.14 Flatten

Explodes **arrays and complex nested types** in hierarchical data (JSON, XML) into flat rows and columns. Essential for nested JSON from APIs.

HANDS-ON LABS

- Lab 1** Load a JSON file where each row has an array field (e.g., order.items[]) — use Flatten to explode items into separate rows
- Lab 2** Flatten a nested struct (e.g., customer.address.city) — map nested fields to flat columns
- Lab 3** Flatten a two-level nested JSON: order → items[] → attributes[] — verify correct row explosion at each level
- Lab 4** After Flatten, add a Filter to remove null exploded rows that result from empty arrays
- Lab 5** Flatten an API response JSON from a REST source — write flattened rows to a SQL table

2.2.15 Flowlet

Reusable sub-graph of transformations. Encapsulate common logic (address cleansing, date normalization) as a callable component — change once, applies everywhere.

HANDS-ON LABS

- Lab 1** Create a Flowlet that trims and uppercases a name column — call it from two different Data Flows
- Lab 2** Create a Date Normalization Flowlet: convert various date string formats to yyyy-MM-dd — reuse across 3 Data Flows
- Lab 3** Add a parameter to a Flowlet — pass the column name to transform as a parameter from the calling Data Flow
- Lab 4** Update the Flowlet logic in one place — verify both Data Flows that call it now use the updated logic
- Lab 5** Create an Audit Columns Flowlet that adds LoadDate, LoadBy, SourceSystem to any dataset — call from 5 Data Flows

2.2.16 Validate Schema & Schema Drift

Schema drift allows Data Flows to process sources whose columns change over time without failing. **Validate schema** rejects unexpected changes.

HANDS-ON LABS

- Lab 1** Enable Schema Drift on a CSV source — add an extra column to the CSV and re-run to verify it processes without failure
- Lab 2** Enable Validate Schema — add an extra column and re-run to observe the validation failure error message
- Lab 3** Use `byName()` expression to reference a drifted column dynamically without hardcoding it in Derived Column
- Lab 4** Use `isNull(byName('newColumn'))` in a Filter to handle rows where the drifted column is absent in older files
- Lab 5** Enable Schema drift on both source and sink — test adding and removing columns; verify end-to-end drift handling

2.2.17 Remove Duplicates using Mapping Data Flows

Pattern using **Aggregate** (group by key, take first()) or **Rank** (row_number=1) to deduplicate — no PySpark code needed.

HANDS-ON LABS

- Lab 1** Use Aggregate: group by customer_id, take first(name), first(email) — deduplicate customer records
- Lab 2** Use Rank: partition by order_id, order by modified_date desc, filter rn=1 — keep most recent order version
- Lab 3** Deduplicate a source with 1M rows — measure row count before and after deduplication transformation
- Lab 4** Use hashAll() to detect true duplicates (all columns identical) — filter where row hash matches a previous row
- Lab 5** Write deduplicated output to ADLS as Parquet — compare file size and row count with the original source

2.2.18 SCD in Data Flows — Type 1 (Overwrite)

No history kept — update records in place. Use **Alter Row** to mark rows as Insert or Update, write to SQL sink with Upsert policy and key columns defined.

HANDS-ON LABS

- Lab 1** Build SCD Type 1: CSV source → Exists (match on customer_id) → Alter Row (insert/update) → SQL sink with Upsert
- Lab 2** Run the pipeline — verify new customers are inserted and existing customers are updated in SQL
- Lab 3** Modify 3 existing rows in the source CSV — re-run and verify only those 3 rows are updated in SQL
- Lab 4** Add a Delete condition in Alter Row — mark rows as delete and configure sink to allow delete operations
- Lab 5** Add a LastModifiedDate Derived Column set to currentTimestamp() — verify it updates on each upsert

2.2.19 SCD in Data Flows — Type 2 (Full History)

Full history with EffectiveStartDate, EffectiveEndDate, IsCurrent. Pattern: **Exists** → **Conditional Split** → **Derived Column** → **Union** → **Sink**.

HANDS-ON LABS

- Lab 1** Build the SCD Type 2 Data Flow: source CSV → Exists (find new/changed) → Conditional Split (new vs changed vs unchanged)
- Lab 2** Add Derived Column for new rows: EffectiveStartDate=currentDate(), EffectiveEndDate=null, IsCurrent=true
- Lab 3** For changed rows: generate two rows — one expiring the old record (IsCurrent=false, EndDate=today) and one inserting the new version
- Lab 4** Union all streams (new inserts + old record expiries) → SQL sink with Insert policy
- Lab 5** Change 2 customer records in source CSV — rerun — verify SQL now has both old (IsCurrent=false) and new (IsCurrent=true) versions

Sample Project — Full Data Warehouse Load with SCD Type 2

CAPSTONE PROJECT

Section 2.2 · End-to-end warehouse fact and dimension load using 8 transformation types

Build a complete data warehouse ETL using Mapping Data Flows: load a Fact_Orders table (incremental) and a Dim_Customer table (SCD Type 2 history tracking) from raw CSV sources — the most common real-world ADF Data Flow pattern.

- Step 1 Dim_Customer SCD Type 2 Flow:** CSV source → Exists → Conditional Split (new/changed/unchanged) → Derived Column (dates + IsCurrent) → Union → SQL sink
- Step 2 Fact_Orders Incremental Flow:** CSV source → Filter (date > watermark) → Join (Dim_Customer on customer_id) → Aggregate (daily totals) → ADLS Parquet sink
- Step 3** Use Derived Column to add audit columns (LoadDate, SourceSystem) to both flows using a shared Flowlet
- Step 4** Orchestrate both flows in one pipeline: Dim load first (synchronous) → Fact load second — schedule with Tumbling Window Trigger
- Step 5** Run for 3 consecutive days — verify SCD Type 2 history builds correctly in Dim_Customer and Fact_Orders grows incrementally

+91 98765
43210

info@psquareacademy.com

www.psquareacademy.com

India

P² Academy

Azure Data Engineering — ADF Only · Track 1 of 4 · Pranav Kumar Reddy · Microsoft Certified Fabric Engineer

Upcoming: **Track 2** Azure DE + Databricks | **Track 3** Azure DE + Synapse | **Track 4** Fabric Data Engineer

Skills Today. Success Tomorrow. · Learn. Build. Deploy. Get Hired. · © 2026 P² Academy · All course content and materials are proprietary